

SQL DQL — SELECT, WHERE, ORDER BY, AGGREGATES, GROUP BY, JOIN

DQL Worksheet — SELECT, WHERE, JOIN (40 Questions)

Tool: VS Code + SQLite

Scenario: Peak Fitness Gym

Prerequisite: DDL + data inserted

INSTRUCTIONS

Make sure gym_app.db has all 5 tables created (DDL module) AND the sample data inserted (shown below) before starting. Write your SQL in dql_exercises.sql. Run with Shift+Enter in the SQLite3 Editor Query Editor (yy0931 extension). Compare your output to what you expect.

PART A — Table Structures (what columns each table must have)

YOUR TASK

Study each table definition below. Using the column names, data types, and constraints shown, write the CREATE TABLE statements yourself in VS Code. Run them in order (trainers first, orders last). Do NOT copy code from the internet — work from these tables.

CREATE order — parents first:

1. trainers (no foreign keys — create first)
2. members (no foreign keys — create second)
3. classes (FK → trainers)
4. bookings (FK → members + classes)
5. payments (FK → members)

DROP order — children first:

1. payments (references members)
2. bookings (references members + classes)
3. classes (references trainers)
4. members (referenced by bookings + payments)
5. trainers (referenced by classes)

Table 1 — trainers

Column Name	Data Type	Constraints	What it stores / Notes
trainer_id	INTEGER	PRIMARY KEY AUTOINCREMENT	Auto-assigned ID. Never include in INSERT.
trainer_name	TEXT	NOT NULL	Full name of the trainer. Required.
email	TEXT	NOT NULL UNIQUE	Email address. Required and must be unique.
speciality	TEXT	(optional)	Subject area e.g. 'Yoga & Pilates'. Can be NULL.
hourly_rate	REAL	CHECK (hourly_rate > 0)	Pay per hour as a decimal. Must be greater than 0.
hire_date	TEXT	NOT NULL	Date hired. Store as text: 'YYYY-MM-DD'.

DDL NOTES

- trainer_id is AUTOINCREMENT — never list it in an INSERT statement
- speciality and hourly_rate are optional (can be NULL or omitted from INSERT)
- Text values need single quotes | Numbers (INTEGER, REAL) do NOT need quotes
- First table to create — no foreign keys

Table 2 — members

Column Name	Data Type	Constraints	What it stores / Notes
member_id	INTEGER	PRIMARY KEY AUTOINCREMENT	Auto-assigned ID. Never include in INSERT.
full_name	TEXT	NOT NULL	Member's full name. Required.
email	TEXT	NOT NULL UNIQUE	Email address. Required and must be unique.
phone	TEXT	(optional — nullable)	Phone number. Can be NULL if not provided.
membership	TEXT	DEFAULT 'basic'	'basic', 'gold', or 'premium'. Defaults to basic.

DQL Worksheet — SELECT, WHERE, JOIN

Peak Fitness Gym · VS Code + SQLite (yy0931) · MySQL awareness throughout

STUDENT COPY

40 Questions

join_date	TEXT	NOT NULL	Date joined. Store as 'YYYY-MM-DD'.
is_active	INTEGER	DEFAULT 1	1 = active member. 0 = inactive. Defaults to 1.
loyalty_pts	INTEGER	DEFAULT 0	Loyalty points balance. Starts at 0.

DDL NOTES

- phone has no NOT NULL — it can be NULL (Carol and Grace have no phone in our data)
- membership defaults to 'basic' — safe to omit from INSERT for new standard members
- is_active and loyalty_pts both have DEFAULT values — omit them to use the defaults

Table 3 — classes

Column Name	Data Type	Constraints	What it stores / Notes
class_id	INTEGER	PRIMARY KEY AUTOINCREMENT	Auto-assigned ID. Never include in INSERT.
class_name	TEXT	NOT NULL	Name of the class e.g. 'Yoga Flow'. Required.
trainer_id	INTEGER	NOT NULL FOREIGN KEY → trainers	Links to trainers.trainer_id. Trainer must exist first.
day	TEXT	NOT NULL	Day of week e.g. 'Monday', 'Tuesday'.
start_time	TEXT	NOT NULL	Time as text e.g. '09:00', '18:00'.
capacity	INTEGER	DEFAULT 20	Maximum students. Defaults to 20 if omitted.

DDL NOTES

- trainer_id is a FOREIGN KEY — the trainer must exist in the trainers table before you can add a class
- PRAGMA foreign_keys = ON must be at the top of your SQL file for FK checks to work
- capacity has a DEFAULT — omit it from INSERT to get 20 automatically

Table 4 — bookings

Column Name	Data Type	Constraints	What it stores / Notes
booking_id	INTEGER	PRIMARY KEY AUTOINCREMENT	Auto-assigned ID. Never include in INSERT.
member_id	INTEGER	NOT NULL FOREIGN KEY → members	Links to members.member_id. Member must exist.
class_id	INTEGER	NOT NULL FOREIGN KEY → classes	Links to classes.class_id. Class must exist.
booking_date	TEXT	NOT NULL	Date of booking. Store as 'YYYY-MM-DD'.
status	TEXT	DEFAULT 'confirmed'	'confirmed' or 'cancelled'. Defaults to confirmed.
attended	INTEGER	DEFAULT 0	0 = did not attend. 1 = attended. Defaults to 0.

DDL NOTES

- Two FOREIGN KEYS: member_id and class_id — both referenced tables must exist before inserting bookings
- status defaults to 'confirmed' — omit it from INSERT when creating a new booking
- attended defaults to 0 — update it to 1 after the class has taken place

Table 5 — payments

Column Name	Data Type	Constraints	What it stores / Notes
payment_id	INTEGER	PRIMARY KEY AUTOINCREMENT	Auto-assigned ID. Never include in INSERT.
member_id	INTEGER	NOT NULL FOREIGN KEY → members	Links to members.member_id. Member must exist.
amount	REAL	NOT NULL CHECK (amount > 0)	Payment amount in pounds e.g. 49.99. Must be positive.
pay_date	TEXT	NOT NULL	Date of payment. Store as 'YYYY-MM-DD'.

DQL Worksheet — SELECT, WHERE, JOIN

Peak Fitness Gym · VS Code + SQLite (yy0931) · MySQL awareness throughout

STUDENT COPY

40 Questions

method	TEXT	DEFAULT 'card'	'card', 'cash', or 'transfer'. Defaults to card.
--------	------	----------------	--

DDL NOTES

- amount is REAL — insert without quotes: 49.99 not '49.99'
- CHECK (amount > 0) means an INSERT with amount = 0 or negative will be rejected
- method defaults to 'card' — omit it from INSERT for card payments

PART B — Starting Data (what rows to insert into each table)

YOUR TASK

Using the data tables below, write the INSERT statements for all 5 tables in your SQL file. Insert them in the same order as the CREATE statements (trainers first, payments last). Run after your CREATE TABLE statements and verify with SELECT COUNT(*) FROM each table.

REMEMBER

- Text values → single quotes e.g. 'Alice Brown' | Numbers → no quotes e.g. 240 or 49.99
- NULL → write NULL without quotes | Omitting a column → its DEFAULT value is used automatically
- Insert in FK order: trainers → members → classes → bookings → payments
- PRAGMA foreign_keys = ON; must be the first line of your SQL file

Insert into trainers — 3 rows

trainer_name	email	speciality	hourly_rate	hire_date
Ms Priya Patel	priya@peakfitness.com	Yoga & Pilates	28.00	2022-03-10
Mr James Lee	james@peakfitness.com	HIIT & Spinning	32.50	2021-06-15
Ms Fatima Hassan	fatima@peakfitness.com	Boxing & Strength	30.00	2023-01-20

HINTS

- Do NOT include trainer_id — it is AUTOINCREMENT and assigned automatically (will become 1, 2, 3)
- Text columns → single quotes e.g. 'Ms Priya Patel'
- hourly_rate is REAL — no quotes e.g. 28.00
- You can insert all 3 rows in one INSERT statement using: VALUES (row1), (row2), (row3);

Insert into members — 8 rows

full_name	email	phone	membership	join_date	loyalty_pts
Alice Brown	alice@email.com	07700111222	premium	2024-01-15	240
Bob Smith	bob@email.com	07700222333	basic	2024-03-20	55
Carol White	carol@email.com	NULL	gold	2024-02-10	180
Dan Jones	dan@email.com	07700444555	basic	2024-04-05	20
Eve Clarke	eve@email.com	07700555666	premium	2024-01-28	310
Frank Osei	frank@email.com	07700666777	gold	2024-05-12	95
Grace Kim	grace@email.com	NULL	basic	2024-06-01	10
Harry Singh	harry@email.com	07700888999	premium	2024-02-20	175

HINTS

- is_active column NOT shown — omit it and the DEFAULT value (1) is used automatically for all 8 members
- Carol and Grace have NULL phone — write NULL without quotes in your INSERT
- membership and join_date are text → single quotes | loyalty_pts is INTEGER → no quotes
- member_id is AUTOINCREMENT — do not include it. Values will be assigned 1 through 8.

Insert into classes — 5 rows

class_name	trainer_id	day	start_time	capacity
Yoga Flow	1	Monday	09:00	15
HIIT Blast	2	Tuesday	18:00	20
Pilates Core	1	Wednesday	10:00	12
Spinning Power	2	Thursday	17:30	18

DQL Worksheet — SELECT, WHERE, JOIN

Peak Fitness Gym · VS Code + SQLite (yy0931) · MySQL awareness throughout

STUDENT COPY

40 Questions

Boxing Fit

3

Friday

07:00

10

HINTS

- `trainer_id` is an INTEGER foreign key — no quotes e.g. 1 not '1'
- `trainer_id` 1 = Ms Patel, 2 = Mr Lee, 3 = Ms Hassan (their IDs from the trainers INSERT above)
- Must insert trainers BEFORE classes, otherwise the FK check will fail
- `class_id` will be auto-assigned 1–5

Insert into bookings — 10 rows

member_id	class_id	booking_date	status	attended
1 (Alice)	1 (Yoga Flow)	2026-06-02	confirmed	1
1 (Alice)	3 (Pilates Core)	2026-06-04	confirmed	1
2 (Bob)	2 (HIIT Blast)	2026-06-03	confirmed	1
3 (Carol)	1 (Yoga Flow)	2026-06-02	confirmed	0
3 (Carol)	4 (Spinning Power)	2026-06-05	confirmed	1
4 (Dan)	2 (HIIT Blast)	2026-06-03	cancelled	0
5 (Eve)	5 (Boxing Fit)	2026-06-06	confirmed	1
5 (Eve)	2 (HIIT Blast)	2026-06-03	confirmed	1
6 (Frank)	4 (Spinning Power)	2026-06-05	confirmed	0
8 (Harry)	1 (Yoga Flow)	2026-06-02	confirmed	1

HINTS

- `member_id` and `class_id` are INTEGER foreign keys — use the numbers only (1, 2, 3...) not the names in brackets
- Both members AND classes must be inserted first — bookings has two FK checks
- Dan's booking (row 6) is 'cancelled' with `attended` = 0 — the only cancelled booking in the dataset
- Grace (`member_id` = 7) has NO bookings at all — she is deliberately absent from this table
- `attended` is INTEGER: 0 = did not attend, 1 = attended. No quotes.

Insert into payments — 6 rows

member_id	amount	pay_date	method
1 (Alice)	49.99	2026-06-01	card
2 (Bob)	19.99	2026-06-01	cash
3 (Carol)	34.99	2026-06-01	card
5 (Eve)	49.99	2026-06-01	card
6 (Frank)	34.99	2026-06-02	transfer
8 (Harry)	49.99	2026-06-02	card

HINTS

- Dan (`id`=4) and Grace (`id`=7) have NO payments — they are deliberately absent from this table
- `amount` is REAL — no quotes e.g. 49.99 not '49.99'
- `member_id` is an INTEGER foreign key — use numbers only (1, 2, 3...)
- `method` defaults to 'card' — you can omit it for the card payments and rely on the DEFAULT

Quick Reference — Key Facts to Know Before Answering Questions

Trainers: Patel (`id`=1) → Yoga Flow + Pilates Core | Lee (`id`=2) → HIIT Blast + Spinning Power | Hassan (`id`=3) → Boxing Fit

Members: 3 basic (Bob, Dan, Grace) · 2 gold (Carol, Frank) · 3 premium (Alice, Eve, Harry)

NULL phone: Carol (`id`=3) and Grace (`id`=7) · Zero bookings: Grace only · Cancelled booking: Dan/HIIT (`booking_id`=6)

Points: Total = 1085 · Average = 135.6 · Min = 10 (Grace) · Max = 310 (Eve)

Payments: Alice, Bob, Carol, Eve, Frank, Harry paid · Dan and Grace have NO payment record

SECTION 1 — SELECT Basics (Questions 1–6)

Q1 SELECT all columns from trainers

Starter

Task:

Write a query that returns every column for every row in the trainers table.

Q2 SELECT specific columns from members

Starter

Task:

Write a query that returns ONLY full_name, membership and loyalty_pts from the members table (not all 8 columns).

Q3 AS alias — friendly column names

Starter

Task:

Rewrite this query so the output columns are labelled 'Member Name' and 'Points':

```
SELECT full_name, loyalty_pts FROM members;
```


Q4 DISTINCT — unique membership types

Starter

Task:

Write a query that lists each membership type ONCE, even though several members share the same plan.

Q5 Concatenation with ||

Intermediate

Task:

Write a query producing one combined column called 'Summary' showing each member's name followed by their plan in brackets, e.g. 'Alice Brown (premium)'.

Q6 Column order in output

Starter

Task:

A colleague wrote: `SELECT full_name, email FROM members;`

But the manager wants email to appear BEFORE full_name in the results. Rewrite the query.

SECTION 2 — WHERE Clause (Questions 7–16)**Q7 Exact match — premium members**

Starter

Task:

Write a query showing full_name and loyalty_pts for members whose membership is exactly 'premium'.

Expected: 3 rows — Alice (240), Eve (310), Harry (175).

Q8 AND — two conditions

Intermediate

Task:

Write a query for members who are 'premium' AND have more than 200 loyalty points.

Expected: 2 rows — Alice (240) and Eve (310). Why is Harry excluded?

Q9 OR — either condition

Intermediate

Task:

Write a query for members who have FEWER than 30 points OR MORE than 200 points.

Expected: 3 rows — Dan (20), Eve (310), Grace (10).

Q10 LIKE — starts with a letter

Intermediate

Task:

Write a query showing all members whose full_name starts with the letter 'A'.

Expected: 1 row — Alice Brown.

Q11 LIKE — contains a substring

Intermediate

Task:

Write a query showing all members whose full_name CONTAINS the letters 'ar' anywhere in the name.

Expected: 2 rows — Carol White and Harry Singh.

Q12 BETWEEN — inclusive range

Intermediate

Task:

Write a query showing members whose loyalty_pts is between 50 and 200 (inclusive of both boundary values).

Expected: 4 rows — Bob (55), Carol (180), Frank (95), Harry (175).

Q13 IN — match a list of values

Intermediate

Task:

Write a query showing members whose membership is EITHER 'gold' OR 'premium', using the IN operator (not OR).

Expected: 5 rows — Alice, Carol, Eve, Frank, Harry.

Q14 IS NULL — find missing data

Intermediate

Task:

Write a query showing members who do NOT have a phone number recorded.

Expected: 2 rows — Carol White and Grace Kim.

Q15 Explain why = NULL is wrong

Advanced

Task:

A student writes: WHERE phone = NULL; and gets 0 rows back even though some phones are NULL.

Explain why this happens and write the corrected query.

Q16 Brackets with AND and OR combined

Advanced

Task:

Write a query for members who are EITHER (premium AND have 200+ points) OR (basic AND fewer than 30 points).

Use brackets to make the grouping clear.

Expected: 3 rows — Dan (basic, 20), Eve (premium, 310), Grace (basic, 10).

SECTION 3 — ORDER BY · LIMIT · OFFSET (Questions 17–22)

Q17 ORDER BY DESC — leaderboard

Starter

Task:

Write a query showing full_name and loyalty_pts for ALL members, sorted from HIGHEST points to LOWEST.

Expected first row: Eve Clarke (310). Expected last row: Grace Kim (10).

Q18 ORDER BY ASC — alphabetical

Starter

Task:

Write a query showing all members' full_name sorted alphabetically (A to Z).

Expected first row: Alice Brown. Expected last row: Harry Singh.

Q19 Multi-column ORDER BY

Intermediate

Task:

Write a query that sorts members first by membership (A to Z), and within each membership group, by loyalty_pts (highest to lowest).

Hint: basic comes before gold alphabetically, gold before premium.

Q20 LIMIT — top 3 only

Intermediate

Task:

Write a query showing ONLY the TOP 3 members by loyalty_pts (highest first).

Expected: Eve (310), Alice (240), Carol (180).

Q21 LIMIT and OFFSET — page 2

Advanced

Task:

Your Flask app shows 3 members per page (sorted by loyalty_pts descending).

Write the query for PAGE 2 — skip the first 3 and show the next 3.

Expected: Harry (175), Frank (95), Bob (55).

Q22 Calculate OFFSET for page 3

Advanced

Task:

Using 3 members per page, calculate and write the LIMIT/OFFSET query for PAGE 3.

Show your OFFSET calculation.

Expected: Dan (20), Grace (10) — only 2 rows because 8 members doesn't divide evenly by 3.

SECTION 4 — Aggregate Functions (Questions 23–27)

Q23 COUNT(*) — total rows

Starter

Task:

Write a query that returns the total number of members in the table, labelled 'total_members'.

Expected: 8

Q24 COUNT(column) — skips NULLs

Intermediate

Task:

Write a query showing how many members HAVE a phone number recorded (not NULL).

Explain why the result is 6 not 8.

Expected: 6

Q25 SUM and AVG together

Intermediate

Task:

Write a query showing the total loyalty points across all members AND the average, rounded to 1 decimal place.

Expected: total_pts = 1085, avg_pts = 135.6

Q26 MIN and MAX

Starter

Task:

Write a query showing the lowest and highest loyalty_pts values in one query.

Expected: lowest = 10 (Grace), highest = 310 (Eve)

Q27 All 5 aggregate functions combined

Advanced

Task:

Write ONE query that returns: total members, total points, average points (1dp), lowest points, and highest points — all in one row.

Expected: 8 | 1085 | 135.6 | 10 | 310

SECTION 5 — GROUP BY and HAVING (Questions 28–32)

Q28 GROUP BY — count per category

Intermediate

Task:

Write a query showing how many members belong to each membership plan.

Expected: basic=3, gold=2, premium=3

Q29 GROUP BY with SUM and AVG

Intermediate

Task:

Write a query showing total and average loyalty points for EACH membership plan, sorted by total points (highest first).

Expected: premium (725, 241.7), gold (275, 137.5), basic (85, 28.3)

Q30 HAVING — filter groups

Advanced

Task:

Write a query showing only the membership plans that have 3 OR MORE members.

Expected: basic (3) and premium (3). Gold is excluded because it only has 2.

Q31 Explain WHERE vs HAVING

Advanced

Task:

A student writes: SELECT membership, COUNT(*) FROM members WHERE COUNT(*) >= 3 GROUP BY membership;

This gives an error. Explain why, and write the corrected query.

Q32 Full combo — WHERE, GROUP BY, HAVING, ORDER BY

Advanced

Task:

Write a query that: considers only active members (`is_active = 1`), groups by membership, keeps only groups with total points over 100, and sorts by total points descending.

Expected: premium (725), gold (275), basic excluded (total=85 which is less than 100).

SECTION 6 — Subqueries and JOINS (Questions 33–40)

Q33 Subquery — above-average members

Advanced

Task:

Write a query showing members whose `loyalty_pts` is ABOVE the average for all members.

Do NOT calculate the average manually — use a subquery.

Average = 135.625. Expected: Eve (310), Alice (240), Carol (180), Harry (175).

Q34 INNER JOIN — bookings with names

Intermediate

Task:

Write a query showing booking_id, the member's full_name, and the class_name for every booking.

Use table aliases b, m, c.

Expected: 10 rows. Grace is excluded because she has no bookings.

Q35 INNER JOIN + WHERE — confirmed only

Intermediate

Task:

Extend the previous query to show ONLY confirmed bookings (exclude the one cancelled booking).

Expected: 9 rows. Booking_id 6 (Dan/HIIT/cancelled) should not appear.

Q36 LEFT JOIN — all members including those with no bookings

Advanced

Task:

Rewrite the booking query as a LEFT JOIN so ALL 8 members appear, even Grace who has no bookings.

Grace should show NULL for booking_id and status.

Hint: which table goes first in FROM?

Q37 LEFT JOIN + IS NULL — members with NO bookings

Advanced

Task:

Using the LEFT JOIN pattern, write a query that returns ONLY members who have never made a booking.

Expected: 1 row — Grace Kim (basic).

--

Q38 LEFT JOIN + GROUP BY — booking count including zero

Advanced

Task:

Write a query showing every member's name and how many bookings they have made, including members with 0 bookings. Sort highest first. Grace should show 0, not be excluded.

Why must you use `COUNT(b.booking_id)` instead of `COUNT(*)`?

Q39 Multiple JOINS — 3 tables in one query

Advanced

Task:

Write a query showing each booking's member name, class name, AND the trainer's name who teaches that class.

You will need bookings, members, classes, and trainers — that is 3 JOINS.

Hint: the link from classes to trainers is `classes.trainer_id = trainers.trainer_id`.

Q40 JOIN + GROUP BY — most popular trainer

Advanced

Task:

Write a query showing each trainer's name and how many CONFIRMED bookings their classes have received, sorted with the most popular trainer first.

Expected: Ms Patel = 3, Mr Lee = 3, Ms Hassan = 1.

(Dan's cancelled booking is excluded by `WHERE status = 'confirmed'`)

DQL Quick Reference — Print and Keep

```
1  -- SELECT basics
2  SELECT * | col1, col2 FROM table;
3  SELECT col AS 'Alias' FROM table;
4  SELECT DISTINCT col FROM table;
5  col1 || ' (' || col2 || ')'      -- concatenate text
6
7  -- WHERE
8  WHERE col = 'value'   AND n > 0
9  WHERE col LIKE 'A%'   -- starts with A
10 WHERE col LIKE '%ar%' -- contains ar
11 WHERE n BETWEEN 50 AND 200 -- inclusive
12 WHERE col IN ('gold', 'premium')
13 WHERE col IS NULL      -- NEVER = NULL
14
15 -- ORDER BY / LIMIT
16 ORDER BY col DESC, col2 ASC
17 LIMIT 3 OFFSET 6      -- page 3 of 3-per-page
18 -- OFFSET formula: (page - 1) * page_size
19
20 -- Aggregates
21 COUNT(*)      -- counts ALL rows including NULLs
22 COUNT(col)    -- skips NULL values
23 SUM(col)      AVG(col)   MIN(col)   MAX(col)
24 ROUND(AVG(col), 1) -- 1 decimal place
25
26 -- GROUP BY / HAVING
27 GROUP BY col
28 HAVING COUNT(*) >= 3      -- filter groups AFTER aggregation
29 -- WHERE runs BEFORE grouping; HAVING runs AFTER
30
31 -- Correct clause order:
32 SELECT FROM WHERE GROUP BY HAVING ORDER BY LIMIT
33
34 -- INNER JOIN (only matching rows in BOTH tables)
35 FROM bookings b
36 INNER JOIN members m ON b.member_id = m.member_id
37 INNER JOIN classes c ON b.class_id = c.class_id
38
39 -- LEFT JOIN (ALL rows from left table, NULL if no match on right)
40 FROM members m
41 LEFT JOIN bookings b ON m.member_id = b.member_id
42 WHERE b.booking_id IS NULL      -- finds members with NO bookings
43
44 -- COUNT rule for LEFT JOIN:
45 COUNT(b.booking_id) -- correct: skips NULLs, shows 0 for Grace
46 COUNT(*)           -- WRONG: shows 1 for Grace instead of 0
```